ASSIGNMENT 2: QPC IP CHECKER REPORT SUBMISSION

CSC3065 Cloud Computing

NOVEMBER 8, 2025

STUDENT NUMBER: 40407039
Name: Luke Milnes

I certify that the submission is my own work, all sources are correctly attributed, and the contribution of any AI technologies is fully acknowledged.

Contents

Task A: Functions 2

Function One: 2

Function Two: 7

Function Three: 14

Function Four: 19

Task B: Improvements 26

Improvement 1 26

Improvement 2 27

Improvement 3 28

Task C: Proxy 28

Task D: Monitoring 32

RepositoryURL: 32

Monitoring and metrics implemented and works: 32

Critical thinking 32

Anything to highlight 32

Task E: Stateful Saving 33

Task F: Design 37

All references used: 40

Task A 40

Function 1: 40

Function 2: 40

Function 3: 41

Function 4: 41

Task B 42

Improvement 1 42

Improvement 2 42

Improvement 3 42

Task C 42

Task D 42

Task E 42

Task F 43

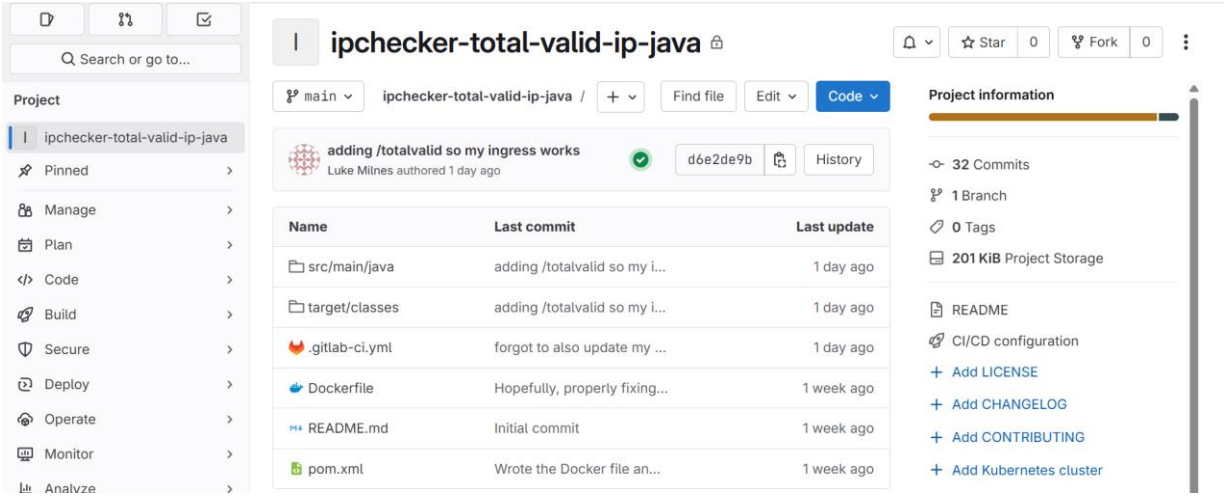
Task A: Functions

Function One:

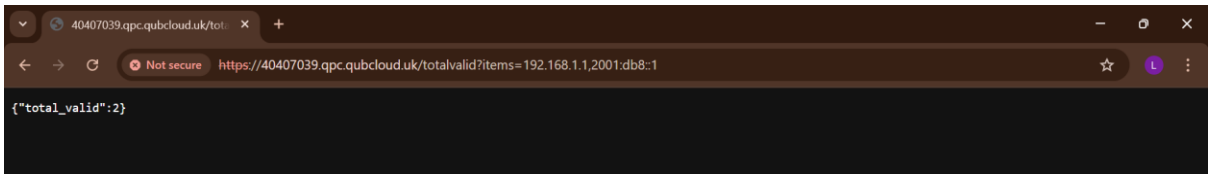
Function(What Operation):

This is the Total valid Ip addresses function- this function should check if IP addresses were valid or not, then count the total valid IP addresses, it should accommodate both IPv4 and IPv6 addresses and it should decern if an IP address is IPV4 through 4 groups separated by "." Dots and for IPv6 through 2-8 groups all separated by ":" colons.

Repository URL: https://repository.hal.davecutting.uk/40407039/ipchecker-total-valid-ip-java.git

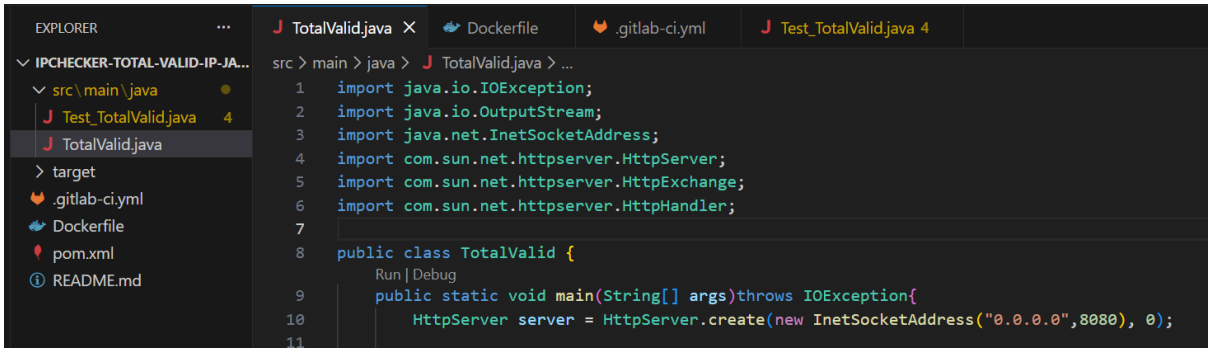


Live Service URL: https://40407039.qpc.qubcloud.uk/totalvalid



Description of implementation (Languages, methods, paradigm, etc):

A different language was advised for each function, so I began my first function with using Java.



I used Java for a multitude of reasons:

It's the language I have the most experience with using over the last couple years including java microservice creations, so I thought it would be a good language to begin with, given that when I have completed this function I could use similar style and structuring for my other functions.

Additionally, java is widely used in a lot of backend cloud systems in industry, meaning that I would most likely have access to a bunch of resources that I could use and to help me build this function, given they would all be appropriately referenced.

Finally, Java has strong testing frameworks, given a lot of the marks were allocated for CI coverage and testing, seemed like a good idea.

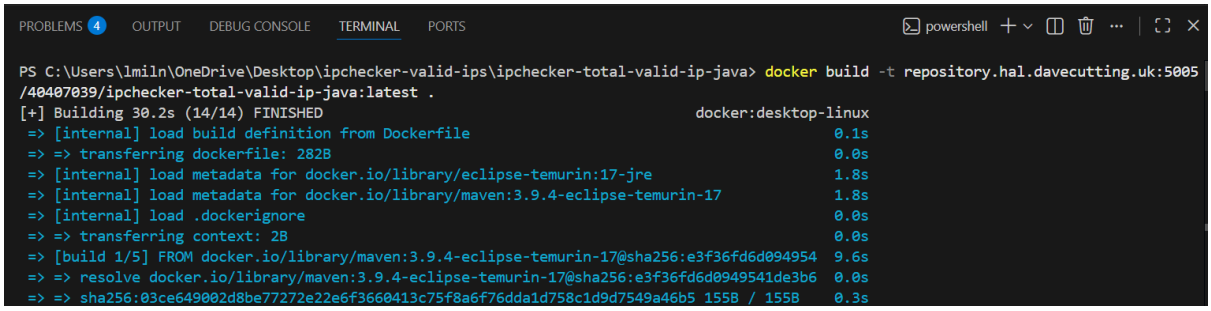
Paradigm:

This function was developed using object orientated Programming, allowing reusability if validation logic, which is important for larger distributed systems.

Docker:

In order to deploy this java microservice in the Queens Private Cloud, a docker file was made in order do build a container image.

Docker Desktop had to be open for these builds and pushes to repo to be successful and the Docker file itself was created using the file was made by replicating the styling shown in practical 2, using WORKDIR, EXPOSE 8080 etc.



The image above shows an example of the docker file build.

Rancher:

After the Docker containers had been pushed to the container registry, it was then uploaded to rancher.

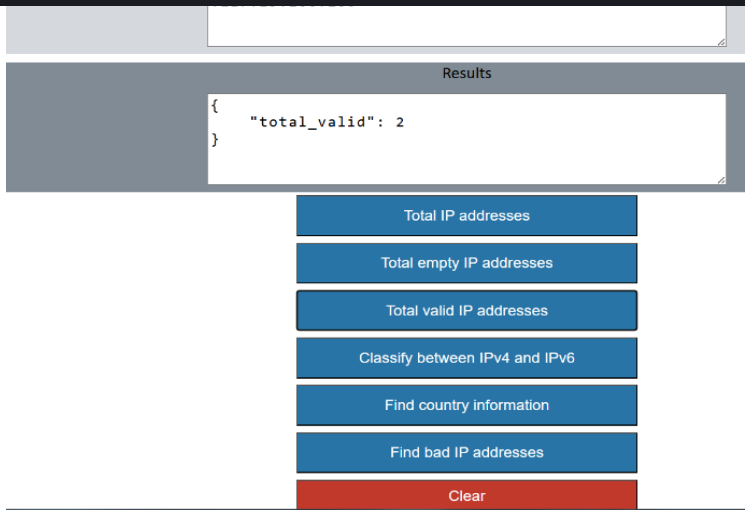
This is important, as this allowed visual monitoring, logs retrieval and confirmation that the service will remain available over time.

The container registry location was fed into rancher and deployed, with a public ingress link tied to them.

Frontend integration:

This service integrates with the existing HTML/JS frontend given through simple fetch quests triggered by the button extensions.

The frontend is uploaded to rancher as a service and I ensured the functions worked correctly.



The image to the right shows functionality of the Total Valid IP address function:

3 Ips have been entered total sperated by commas but the result only shows toal valid to be 2.

This is because only 2 of the 3 Ips I entered are actually valid since the last one is supposed to be IPv4 but has 5 groups instead of 4.

```
98
99
100  // Java function with my Total Valid IPs
101  function getTotalValidIPs() {
102      serviceCall("https://40407039.qpc.qubcloud.uk/totalvalid", "items");
103  }
104
```

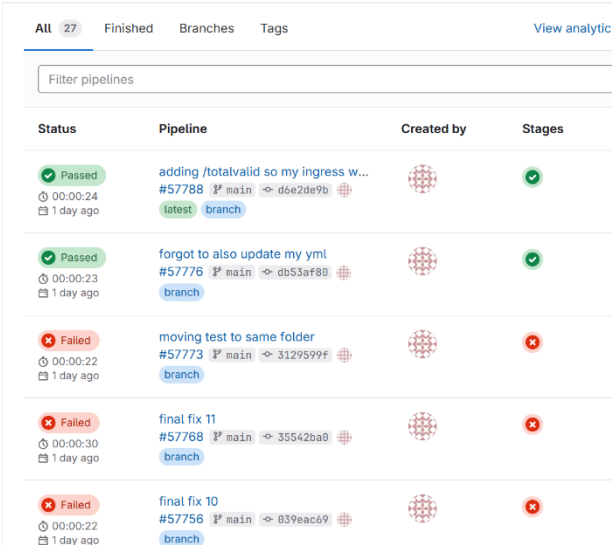
Additonally the frontend code was edited in various places to include the new function including changing the buttons from inactive to active to show that they are now operational.

Description of testing:

Pipelines:

Every time I changed my CI files, and committed and pushed changes to my repository, GitLab would automatically trigger a new pipeline.

These pipelines would run a series of tests on files including dependency installation, building the application and any unit tests.



This allowed me to easily see if the build failed or passed and it allowed me to focus my attention to the exact area where the issue occurred as it outputted a detailed error log of what went wrong and when.

```
23  error: file not found: *.java
24  Usage: javac <options> <source files>
25  use --help for a list of possible options
26  Cleaning up project directory and file based variables   00:00
27  ERROR: Job failed: exit code 1
```

CI Testing:

When it came to CI, I originally had it in my function so that gitlab could install dependencies, build my java Function, and auto run pipelines after each push, ensuring my code compiles successfully and preventing any broken code form being deployed and allowing me to have access to my logs.

When I confirmed all of this I then edited my .gitlab-ci.yml to incorporate my test file.

```java
src > main > java > Test_TotalValid.java > ...
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;

public class Test_TotalValid {
    Run | Debug
    public static void main(String[] args) throws Exception {

        int count = 0;

        if (TotalValid.checkIP(ip: "192.168.1.1")) count++;
        if (TotalValid.checkIP(ip: "2001:db8::1")) count++;

        if (count != 2) {
            throw new Exception("Expected total_valid to be 2");
        }

        System.out.println("Java CI test passed");
    }
```

After writing this automated test file for this function, its role changed.

Originally it would only verify if the project could be successfully built without errors.

Now the test would make the pipeline responsible for executing unit level validation of the function.

```java
    public static boolean checkIP(String ip){
        ip = ip.trim();

        if(ip.contains(".")){
            String[] ipv4 = ip.split("\\.");
            if(ipv4.length == 4){
                return true;
            }
        }
        if(ip.contains(":")){
            String[] ipv6 = ip.split(":");
            if(ipv6.length >= 2 && ipv6.length<=8){
                return true;
            }
```

The test automatically sends IP addresses to the checkIP method in the totalvalid class and checks that all the returned results meet the expected outputs, in this case it would be counting two valid IP addresses.

If the function returns the wrong values etc then the pipeline will fail, letting me know there is an error and preventing further deployment.

Anything else to highlight:

Ai such as Chat gpt were used in this project only for the purpose of clarifying error messages, helping to debug code and identifying issues in my code , it has not been used to directly generate or write the original code in this assignment.

Additionally, I turned to some websites for additional help on certain stages of my functions all of which will be listed under here:

https://docs.oracle.com/javase/8/docs/jre/api/net/httpserver/spec/com/sun/net/httpserver/HttpServer.html

Server class I used

https://docs.oracle.com/javase/8/docs/jre/api/net/httpserver/spec/com/sun/net/httpserver/HttpExchange.html

used for my "exchange.getRequestMethod();" and my "exchange.getResponseBody();"

https://docs.oracle.com/javase/8/docs/api/java/lang/String.html#split-java.lang.String-

where I learned how to split ips

https://developer.mozilla.org/enUS/docs/Learn_web_development/Core/Scripting/JSON

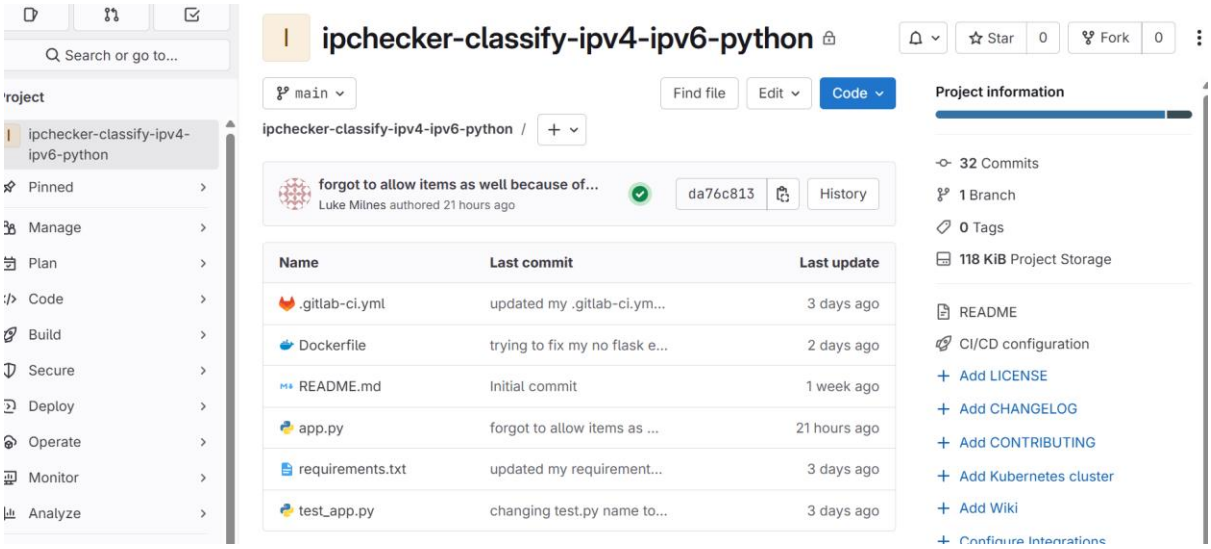
helped with my manually built JSON

Function Two:

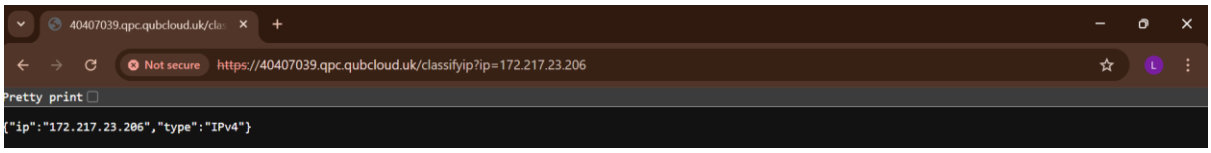
Function(What Operation):

Classify between IPv4 and IPv6- this function should check IP addresses then classify them into IPv4 and IPv6. It should do this through checking if there are 4 groups separated by "." Dots for IPv4 and 2-8 groups separated by ":" colons for IPv6. It should output the IP address again then list what type of address it is alongside it.

Repository URL: https://repository.hal.davecutting.uk/40407039/ipchecker-classify-ipv4-ipv6-python.git



Live Service URL: https://40407039.qpc.qubcloud.uk/classifyip



Description of implementation (Languages, methods, paradigm, etc):

My next chosen language was Python, this was also done for various reasons such as:

The simplicity of Python, Python is known for its simplicity and its ability to allow rapid prototyping without the need for mass amounts of boilerplate code.

Additionally I have also worked with python in terms of microservices before in a previous module alongside java so I already had a bit of experience when it came to writing code in java.

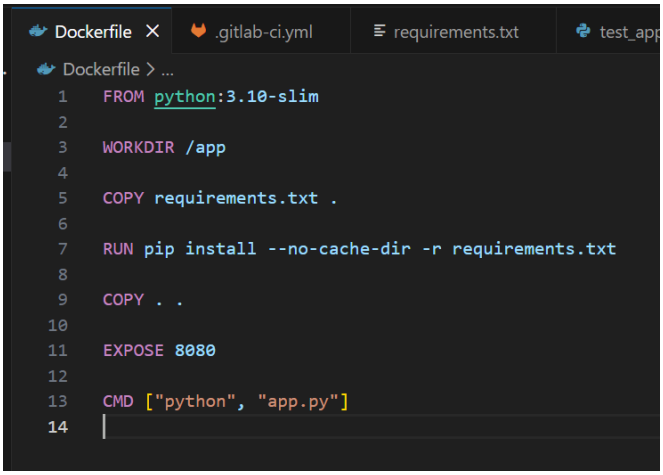
Python is also a very popular language when it come to cloud based microservices, which would make it well suited when it come to containerising services, and since its so popular increases the chances of their being more examples and references online I can learn from for my own assignment.

I chose the Flask micro-framework additionally for its simplicity and speed for smaller web based microservices

This service is similar to my previous java service in the way that I use string-based validation to determine whether an address is an IPV4, IPV6 or is invalid.

The sting and split methods allow this to be done without requiring the use of external libraries.

Overall, I approached this function with how I designed my previous java function in mind.



Paradigm:

Similarly to my previous function this function follows a predominantly Object orientated approach, which can be seen demonstrated through the ClassifyIP class, which is used to encapsulates the logic used to decide if an address is IPV4.IPv6 or Invalid.

Additionally, this structure allows me to keep my overall logic, reuseable, modular and consistent with my previous function.

Docker:

As with my previous function, to deploy this python microservice in the Queens Private Cloud, a docker file would need to be creates to build a container image.

This Docker file similarly follows the styling of the previous functions Docker set up, however, in this instance since I used a requirement.txt file to hold the flask installation, the docker file has to run the install of everything inside the requirements.txt file.

```
PS C:\Users\lmiln\OneDrive\Desktop\ipchecker-classify-ipv4-ipv6\ipchecker-classify-ipv4-ipv6-python> docker build
 -t repository.hal.davecutting.uk:5005/40407039/ipchecker-classify-ipv4-ipv6-python:v3 .
>>
[+] Building 5.3s (10/10) FINISHED                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                0.1s
 => => transferring dockerfile: 214B                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.10-slim                 1.5s
 => [internal] load .dockerignore                                                   0.0s
 => => transferring context: 2B                                                     0.0s
 => [1/5] FROM docker.io/library/python:3.10-slim@sha256:c299e10e0070171113f9a1f109dd05e7e63  0.1s
 => => resolve docker.io/library/python:3.10-slim@sha256:c299e10e0070171113f9a1f109dd05e7e63  0.1s
```

The image above shows an example of me building the docker image.

Rancher:

Similarly to the last function, when I have finished my docker file and built the Docker containers will have been pushed to the container registry, leading to them being uploaded to rancher.

Once again, this is important, as this allowed visual monitoring, logs retrieval and confirmation that the service will remain available over time.

The container repository location is then fed into rancher correctly labelled with the right ports etc and addresses, then deployed and linked with my public ingress link allowing the frontend to call my functions.

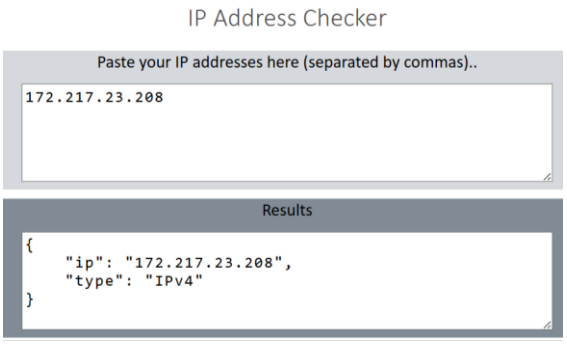
Frontend integration:

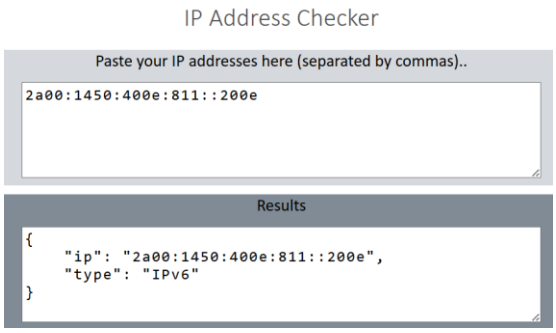
As a reminder, this service integrates with the existing HTML/JS frontend given through simple fetch quests triggered by the button extensions.

The frontend is uploaded to rancher as a service and I ensured the functions worked correctly.

The images to the right show how the Classifier works.

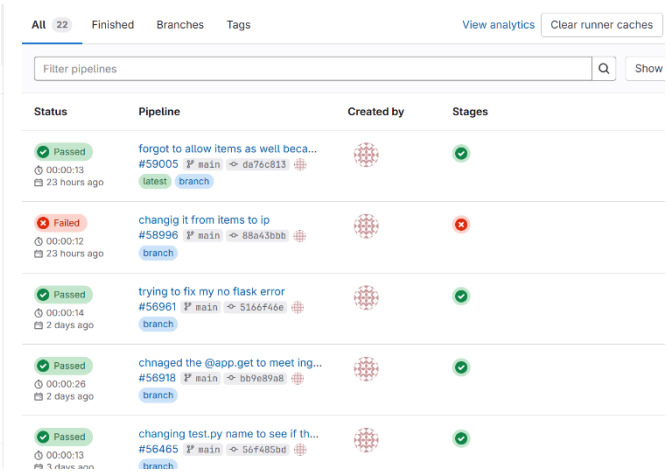
If an IP address with 4 parts seperates by "." Dots is there then its an IPv4 address and if an IP address with 2-8 parts seperated by":" colons then it must be an IPv6 address.





Description of testing:

Pipelines:



Every time I changed my CI files, and committed and pushed changes to my repository, GitLab would automatically trigger a new pipeline.

These pipelines would run a series of tests on files including dependency installation, building the application and any unit tests.

Similarly to my previous function, the pipelines show me when my overall build passes or fails, then outputs a log that I can acces that shows me exactly when and where my build fails and the reason for it exiting, allowing me to fast track my bug fixing.

CI tests:

Like my Java function, I began by making .gitlab-ci.yml in order to download all dependencies and in particular ensure flask was installed correctly, because for a while I had a long standing issue where I couldn't get the docker container to correctly have flask module installed.

```
31 ERROR: Could not find a version that satisfies the requirement flask==3.0
   2 (from versions: 0.1, 0.2, 0.3, 0.3.1, 0.4, 0.5, 0.5.1, 0.5.2, 0.6, 0.6.
   1, 0.7, 0.7.1, 0.7.2, 0.8, 0.8.1, 0.9, 0.10, 0.10.1, 0.11, 0.11.1, 0.12,
   0.12.1, 0.12.2, 0.12.3, 0.12.4, 0.12.5, 1.0, 1.0.1, 1.0.2, 1.0.3, 1.0.4,
   1.1.0, 1.1.1, 1.1.2, 1.1.3, 1.1.4, 2.0.0rc1, 2.0.0rc2, 2.0.0, 2.0.1, 2.0.
   2, 2.0.3, 2.1.0, 2.1.1, 2.1.2, 2.1.3, 2.2.0, 2.2.1, 2.2.2, 2.2.3, 2.2.4,
   2.2.5, 2.3.0, 2.3.1, 2.3.2, 2.3.3, 3.0.0, 3.0.1, 3.0.2, 3.0.3, 3.1.0, 3.
   1.1, 3.1.2)
32 ERROR: No matching distribution found for flask==3.02
33 [notice] A new release of pip is available: 23.0.1 -> 25.3
34 [notice] To update, run: pip install --upgrade pip
35 Cleaning up project directory and file based variables          00:00
36 ERROR: Job failed: exit code 1
```

The image above shows an example pipeline where it wasn't downloading flask because I had made an error in spelling it, calling it flask 3.02 instead of flask 3.0.2.

I then later repurposed my ci tests to also include my test_app.py file.

test_app.py > ...

```python
import app

def test_classify_ipv4():
    result = app.classifier.classify("1.2.3.4")
    assert result == "IPv4"

def test_classify_ipv6():
    result = app.classifier.classify("2a00:1450:400e::200e")
    assert result == "IPv6"

def test_classify_invalid():
    result = app.classifier.classify("not_an_ip")
    assert result == "Invalid"
```

This file was made so I could run some proper CI tests in my .gitlab-ci.yml file to ensure my code worked the way I intended for it to work.

.gitlab-ci.yml

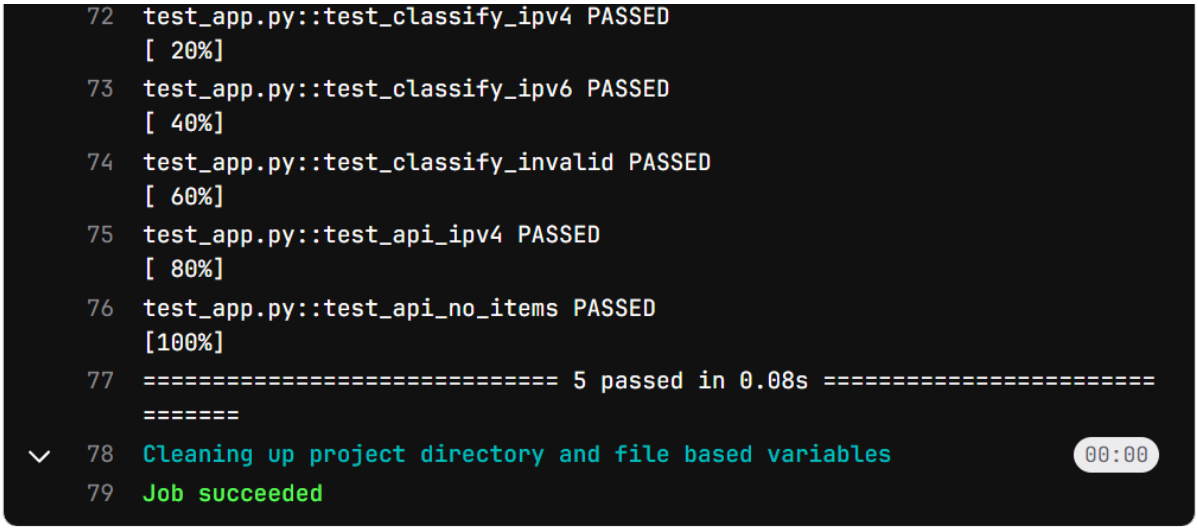
```yaml
image: python:3.10-slim

stages:
  - test

python_tests:
  stage: test
  script:
    - pip install -r requirements.txt
    - pytest -v
```

My final .yml file to test my code above.

When this final test passed in my pipelines then I knew that I had achieved the full functionality for this service that I intended to.



Anything else to highlight:

Ai such as Chat gpt were used in this project only for the purpose of clarifying error messages, helping to debug code and identifying issues in my code , it has not been used to directly generate or write the original code in this assignment.

Additionally, I turned to some websites for additional help on certain stages of my functions all of which will be listed under here:

https://flask.palletsprojects.com/en/stable/quickstart/#routing

This website was used to help me understand how to properly route with flask.

https://flask.palletsprojects.com/en/stable/api/#module-flask.json

was used to help me understand how to use JSON with Flask

https://docs.python.org/3/library/stdtypes.html#str.split

where I got my idea for splitting strings in python.

https://flask.palletsprojects.com/en/stable/testing/

Used for when I was doing my CI tests

https://docs.python.org/3/reference/simple_stmts.html#the-assert-statement

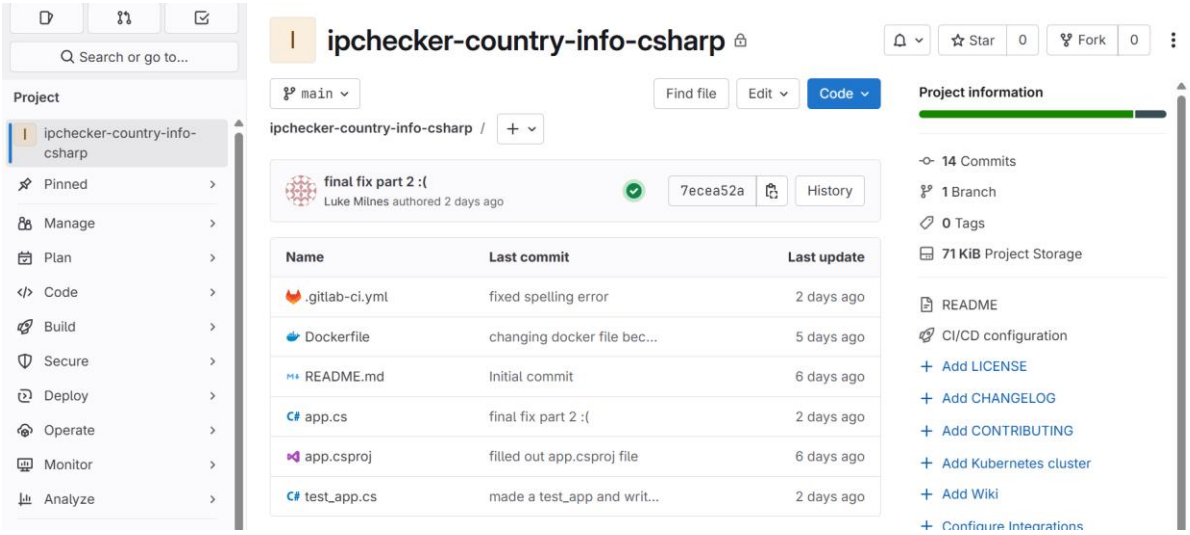
Additionally referred to this in CI testing

Function Three:

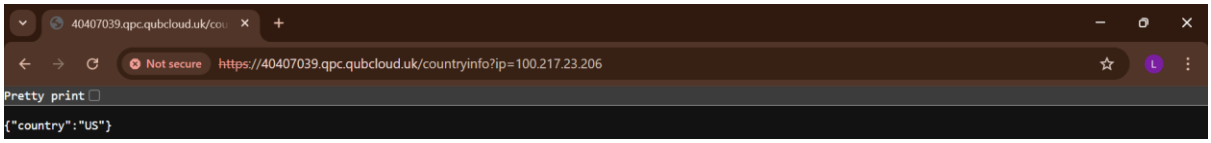
Function(What Operation):

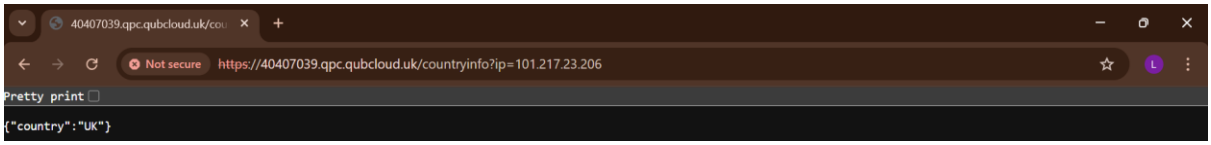
Find country information- this function should check only IPv4 address and then identify what country they belong to based off of the first 3 digit of the IPv4 address ”if the first 3 digits are 100, then the country is US” It should display the IP address again then list what country it belongs to alongside it.

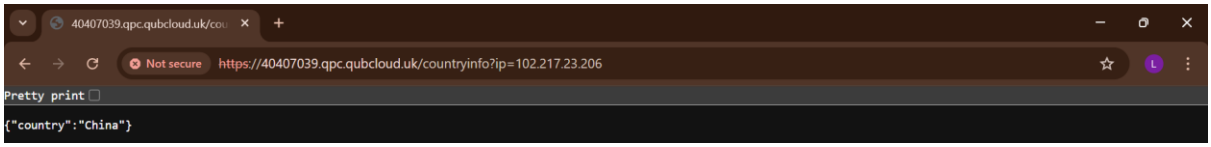
Repository URL: https://repository.hal.davecutting.uk/40407039/ipchecker-country-info-csharp.git

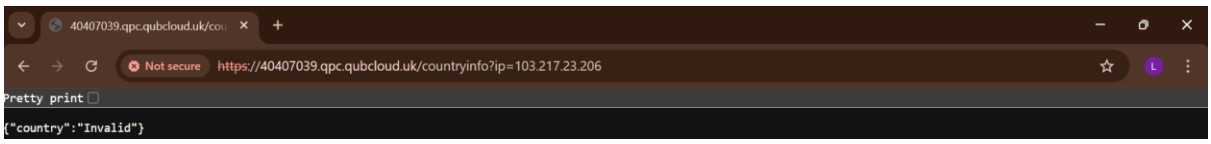


Live Service URL: https://40407039.qpc.qubcloud.uk/countryinfo



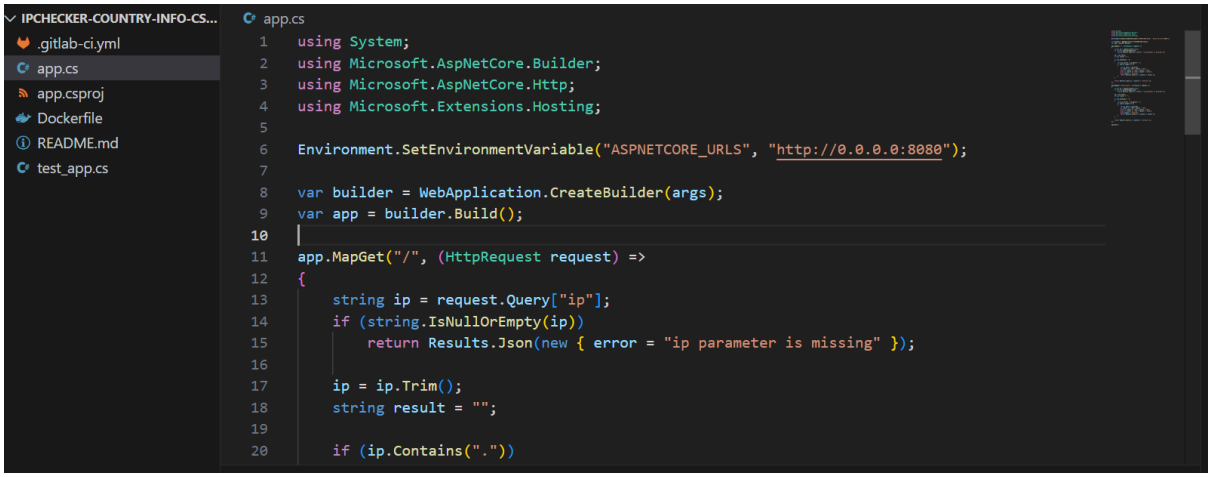






Description of implementation (Languages, methods, paradigm, etc):

My third chosen Language was C#, this was also done for numerous reasons in which I will list:



I chose C# because it is a commonly used language that is used by larger organisations for their cloud environments such as companies that rely on Microsoft azure infrastructure.

This means that even though I have minimal experience using c# as a language, I was fairly confident that there would be a lot of tutorials and references online to help guide me and with my knowledge I had gained through building my python and java functions previously.

The logic chosen for this function remains mostly consistent with my other functions although I chose to receive Ip addresses via a query parameter then perform simple string parsing in order to allow me to extract the firs part of the IPv4 address allowing me to match it to my already known addresses. E.g. 100 for US, 101 for UK and 103 for China etc.

Paradigm:

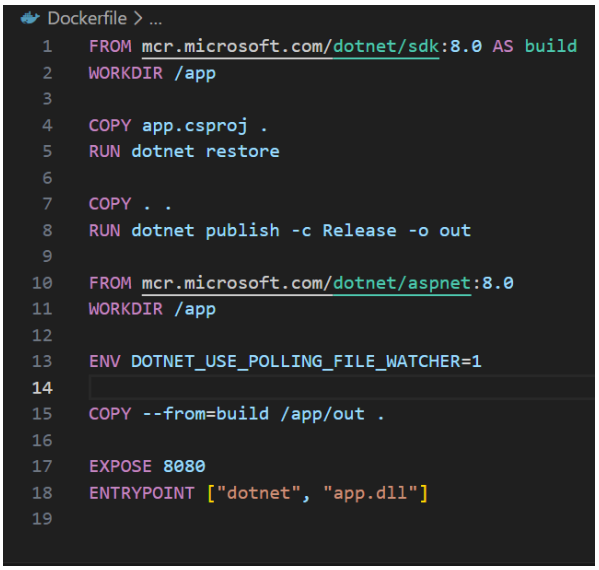
C# usually follows a multi-paradigm approach, commonly object orientated, however, In this case I tried to use a procedural style similarly to my python, to hopefully keep the function lightweight and simple.

Docker:

As with my previous functions, to deploy this python microservice in the Queens Private Cloud, a docker file would need to be creates to build a container image.

This docker file is similar but quite different to my previous functions.

This docker file is noticeable more complex compared to my python function, this is because C# applications must be compiled and



publish before we can execute them, unlike python which you can just run as interpreted scripts.

Using this multistage approach helps reduce the final image size and is usually standard practice for .Net applications.

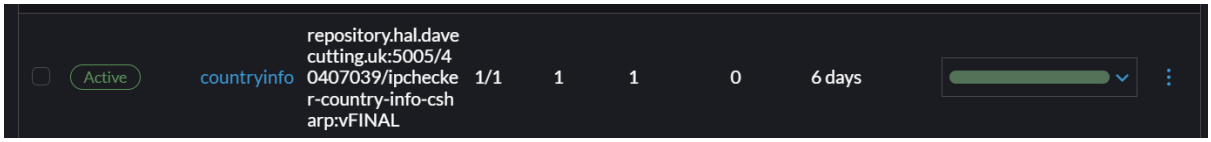
An example of me building my docker image

Rancher:

Similarly to my previous functions, when I have finished my docker file and built the Docker containers will have been pushed to the container registry, leading to them being uploaded to rancher.

Once again, this is important, as this allowed visual monitoring, logs retrieval and confirmation that the service will remain available over time.

The container repository location is then fed into rancher correctly labelled with the right ports etc and addresses, then deployed and linked with my public ingress link allowing the frontend to call my functions.



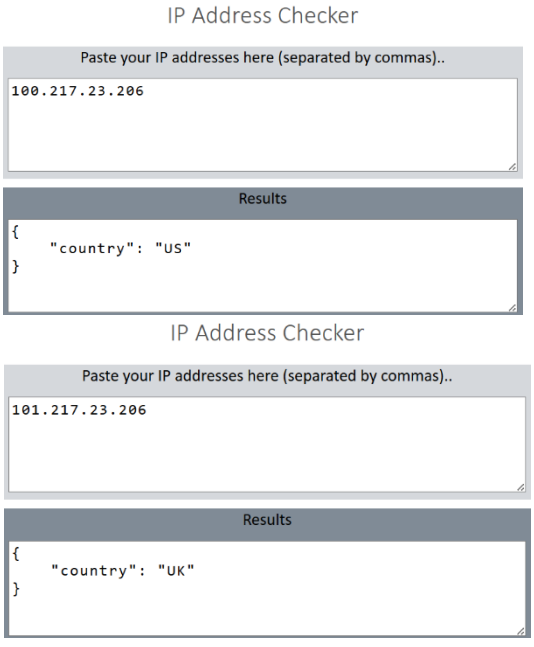
Frontend implementation:

As a reminder, this service integrates with the existing HTML/JS frontend given through simple fetch quests triggered by the button extensions.

The frontend is uploaded to rancher as a service and I ensured the functions worked correctly.

The images on the right show the functionality of the countryinfo function that I wrote.

It reads the first group of the IP address given, separated by "." Dots, then based on that outputs the result as whatever country matches

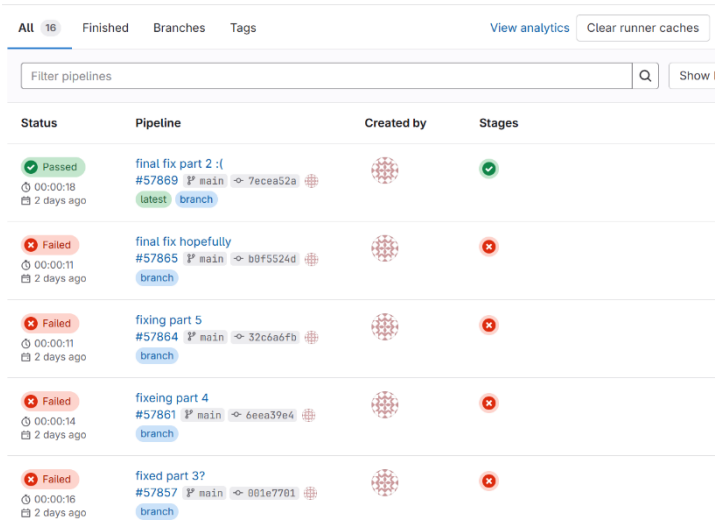


the first group. For example if the first group in the IP is 100 then it will output US, if the first group is 101 then UK and if its 102 then China.

Description of testing:

Pipelines:

Every time I changed my CI files, and committed and pushed changes to my repository, GitLab would automatically trigger a new pipeline.



These pipelines would run a series of tests on files including dependency installation, building the application and any unit tests.

Similarly to my previous functions, the pipelines show me when my overall build passes or fails, then outputs a log that I can acces that shows me exactly when and where my build fails and the reason for it exiting, allowing me to fast track my bug fixing.

CI tests:

Like my Java function and Python function, I began by making .gitlab-ci.yml in order to download all dependencies, this basic .yml only performed basic build validation including a single stage that restored dependencies and confirmed it could build without errors.

However, it couldn't verify if the function behaved correctly when running and as a result could pass the pipeline even if the function technically didn't work at all.

```
54 ms).
$ dotnet build --configuration Release
  Determining projects to restore...
  All projects are up-to-date for restore.
  app -> /builds/40407039/ipchecker-country-info-csharp/bin/Release/net8.
0/app.dll
Build succeeded.
    0 Warning(s)
    0 Error(s)
Time Elapsed 00:00:02.98
Cleaning up project directory and file based variables                00:01
Job succeeded
```

After I updated my CI tests to introduce automatic testing, it allowed the CI pipeline to not only just build but also run it and send out real HTTP requests to verify the output was correct.

test_app.cs

```
using System;
using System.Net.Http;
using System.Threading.Tasks;

class TestApp
{
    static async Task Main()
    {
        using var client = new HttpClient();
        var response = await client.GetStringAsync("http://localhost:8080/?ip=100.217.23.206");

        if (!response.Contains("US"))
        {
            throw new Exception("Expected US for IP starting with 100");
        }

        Console.WriteLine("Test passed");
    }
}

```

This meant that the pipeline now accesses functional accuracy as well not just how well the build went, improving reliability and preventing broken logic from being deployed.

.gitlab-ci.yml

```
image: mcr.microsoft.com/dotnet/sdk:8.0

stages:
  - test

test-job:
  stage: test
  script:
    - dotnet restore
    - dotnet build --configuration Release
    - dotnet run --project app.csproj &
    - sleep 5
    - curl -s "http://localhost:8080/?ip=100.217.23.206" | grep -q "US"
```

Anything else to highlight:

Ai such as Chat gpt were used in this project only for the purpose of clarifying error messages, helping to debug code and identifying issues in my code , it has not been used to directly generate or write the original code in this assignment.

Additionally, I turned to some websites for additional help on certain stages of my functions all of which will be listed under here:

https://learn.microsoft.com/en-us/aspnet/core/fundamentals/minimal-apis?view=aspnetcore-10.0

This is where I got some of my builder lines from for this function e.g. “var builder = WebApplication.CreateBuilder(args);” and ”var app = builder.Build();”

https://learn.microsoft.com/en-us/dotnet/api/system.string.split?view=net-10.0

where I got my string split logic from.

https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/async-scenarios

async for my CI testing

https://learn.microsoft.com/en-us/dotnet/api/system.net.http.httpclient?view=net-10.0

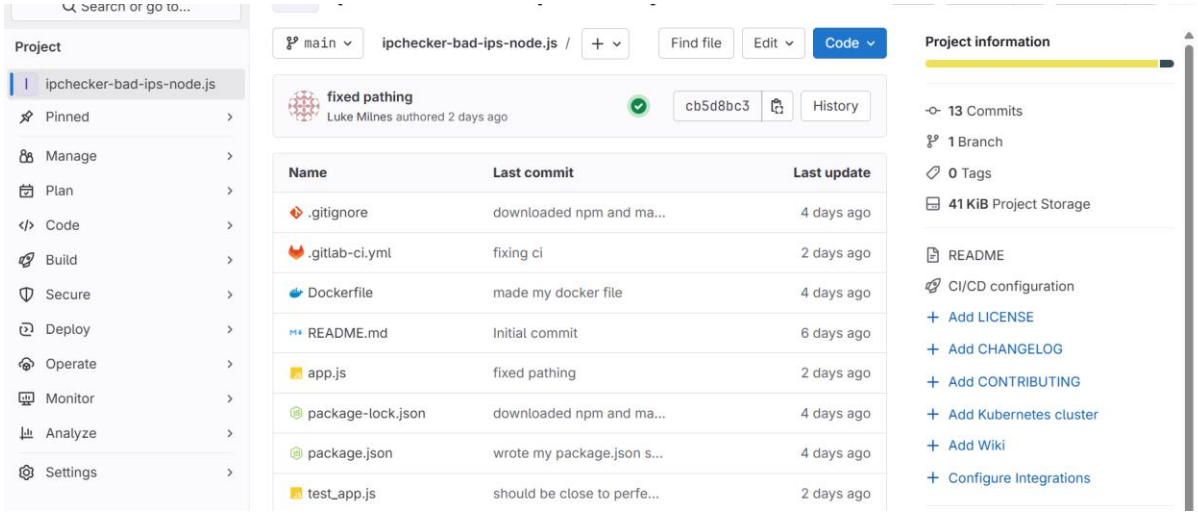
For making my http requests

Function Four:

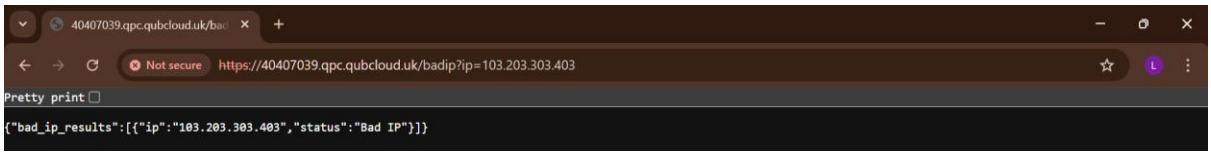
Function(What Operation):

Find bad IP addresses- this function should compare IPv4 addresses given with an already given list of bad IPv4 addresses and then label the IPv4 addresses as “Bad IP” if these 2 IP addresses match. It should output the IPv4 address, then list if it is a good or bad IP alongside it.

Repository URL: https://repository.hal.davecutting.uk/40407039/ipchecker-bad-ips-node.js.git



Live Service URL: https://40407039.qpc.qubcloud.uk/badip



Description of implementation (Languages, methods, paradigm, etc):

Node.js was the final language I chose and here is some of the reasons why:

Node.js is one of the most widely used backend languages for modern cloud development, especially so when it comes to the creation of lightweight API driven systems and particularly for microservices.

It uses a non-blocking execution model which makes it very efficient when it comes to handling a large amount of requests at the same time, this is important in cloud environments where availability, scalability and responsiveness of services is essential.

Additionally JavaScript allows the use of npm allowing me to integrate external libraries with minimal overhead.

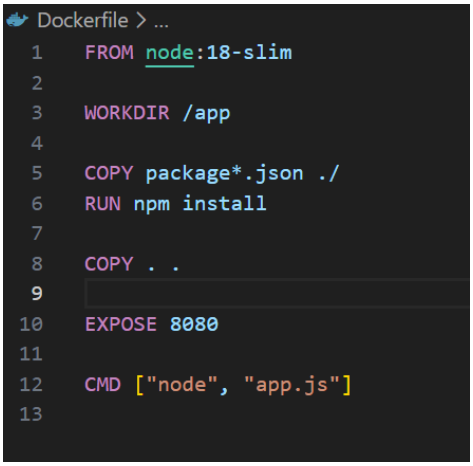
Even though Node.js is not one of my strongest languages, I had already developed 3 functions using different languages at this point and I was pretty confident that I had a consistent service pattern from these functions that I could implement in my node, leading to my code to be fairly similar such as accepting IP addresses as query parameters and splitting the input, to be compared against a list of given bad IP addresses.

Paradigm:

Node.js is a bit different to the previous functions, since they can all be used in both object orientated and procedural programming styles, Node.js is more aligned with non-blocking, asynchronous paradigm, which is pretty popularly adopted in some modern cloud applications.

Docker:

As with my previous functions, to deploy this python microservice in the Queens Private Cloud, a docker file would need to be creates to build a container image.



Compared to previous functions docker file, Node.js doesn't require a heavily complicated docker file In order to run, it just additionally requires an npm install to be run inside it so that the image also contains npm and functions correctly when its in its container as a service.

Rancher:

Similarly to my previous functions, when I have finished my docker file and built the Docker containers will have been pushed to the container registry, leading to them being uploaded to rancher.

Once again, this is important, as this allowed visual monitoring, logs retrieval and confirmation that the service will remain available over time.

The container repository location is then fed into rancher correctly labelled with the right ports etc and addresses, then deployed and linked with my public ingress link allowing the frontend to call my functions.

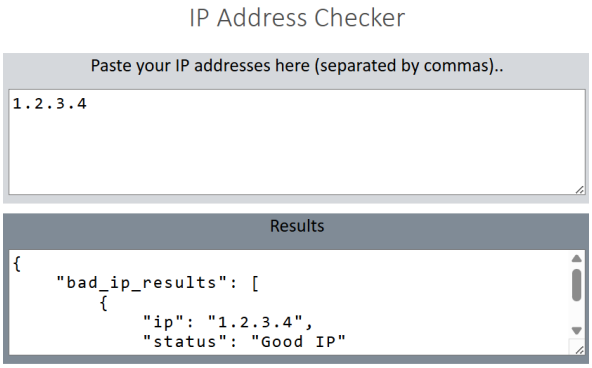
Frontend implementation:

As a reminder, this service integrates with the existing HTML/JS frontend given through simple fetch quests triggered by the button extensions.

The frontend is uploaded to rancher as a service and I ensured the functions worked correctly.

The images to the right showcase the following inputs and outputs from the frontend when its linked to my function.

"101.201.301.401" this IP address is one of the IP addresses listed in my app.js detailing that if they are entered they should return "Bad IP", this function is therefore proven to do that.



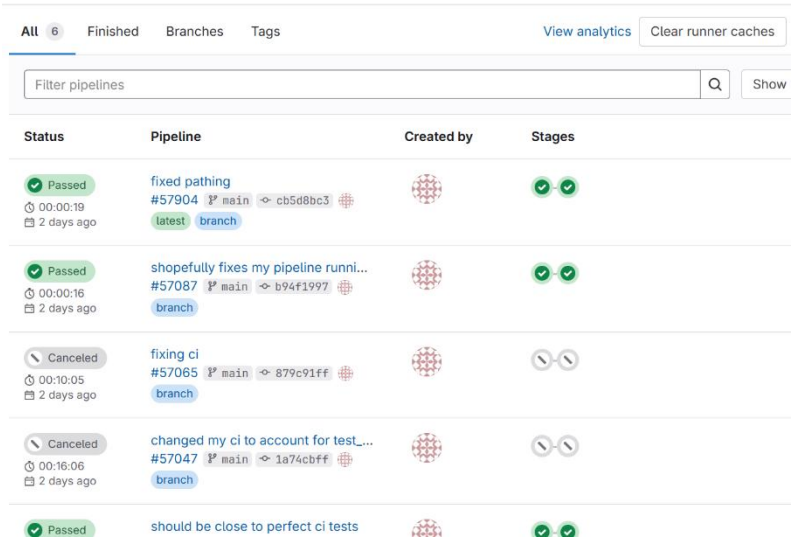
However, this function was also required to output "Good IP" when an IP address not matching the Bad IP addresses was entered, so when 1.2.3.4 was entered it outputted "Good IP" showing that the function works correctly and only returns "Bad IP" for the Bad Ip addresses listed in app.js.

Description of testing:

Pipelines:

Every time I changed my CI files, and committed and pushed changes to my repository, GitLab would automatically trigger a new pipeline.

These pipelines would run a series of tests on files including dependency installation, building the



application and any unit tests.

Similarly to my previous functions, the pipelines show me when my overall build passes or fails, then outputs a log that I can acces that shows me exactly when and where my build fails and the reason for it exiting, allowing me to fast track my bug fixing.

CI Testing:

When I got to building the CI for this function I had already decided that unlike my previous functions, I was going to try and build and test the function at the same time in the CI instead of just building it first then testing it later.

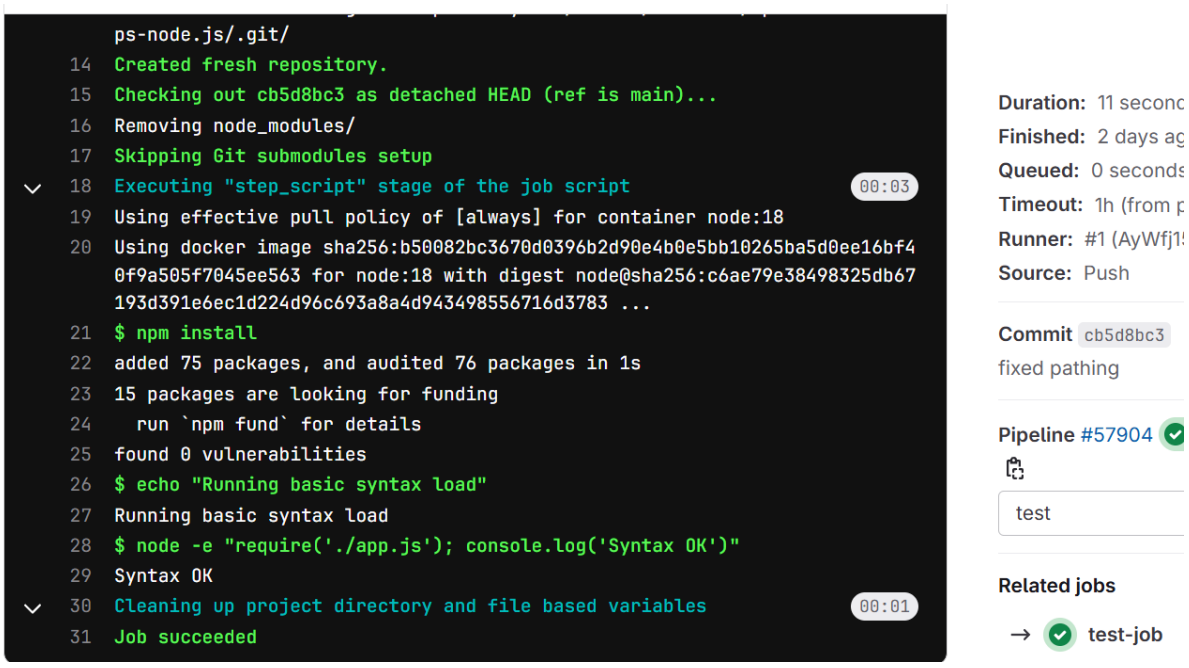


Image of the test stage above

My CI tests ran I the following structure, They had two main stages:

A test stage where the pipeline would download all the dependencies required using the npm install, and executed basic syntax by requiring the file “node -e "require('./app.js'); console.log('Syntax OK')"” to ensure that the code contained no module loading errors and syntax, preventing JavaScript from being deployed if it did not include functional assertions.

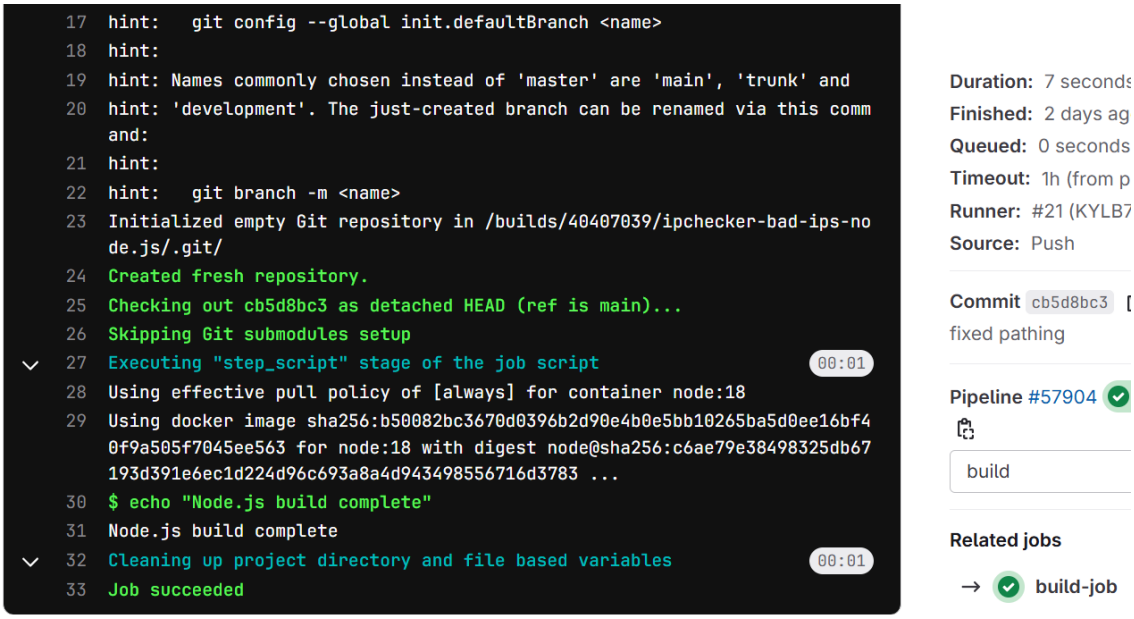


Image of the build stage above.

The build stage simply was used to confirm the pipeline had reached completion successfully without any errors, this reflects more on the early stages of implementing this function where my main focus was on insure the CI executed without failing.

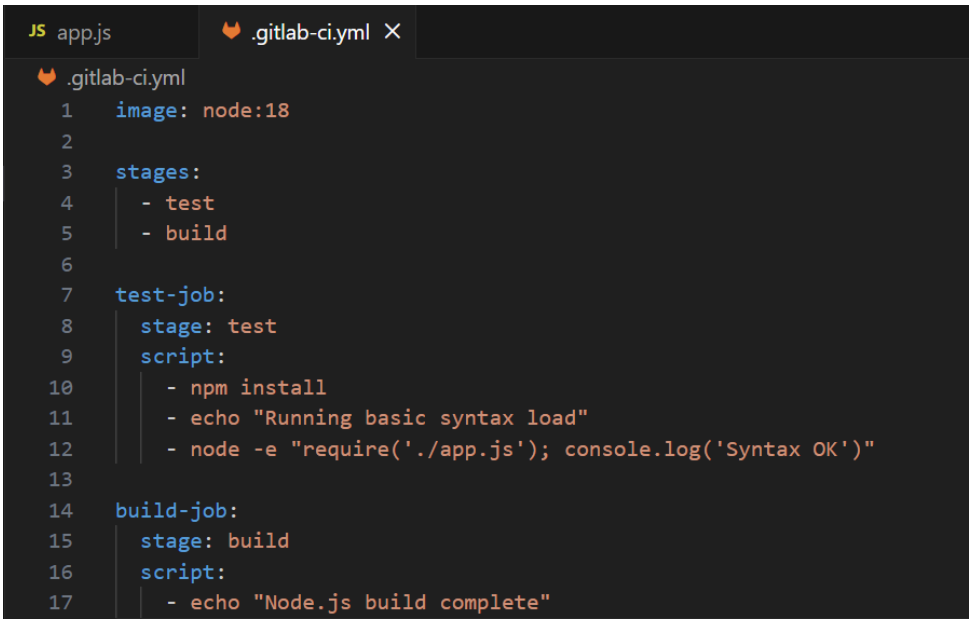


Image above shows what my final CI looks like, including all my stages of code.

Finally to note, my test_app.js file:

This image shows how I checked my function worked as expected before I linked it to my frontend, it clearly tests IP addresses that are set to be classified as Good IP, it tests IP addresses that have been coded to show up as Bad IP, and it shows a mix of Ips and how it would check that the response is equal to what was expected.

Anything else to highlight:

Ai such as Chat gpt were used in this project only for the purpose of clarifying error messages, helping to debug code and identifying issues in my code , it has not been used to directly generate or write the original code in this assignment.

Additionally, I turned to some websites for additional help on certain stages of my functions all of which will be listed under here:

https://expressjs.com/en/guide/routing.html

Used for express routing in my function.

https://expressjs.com/en/api.html#req.query

query parameters help.

https://developer.mozilla.org/enUS/docs/Learn_web_development/Core/Scripting/JSON

Outputting JSON.

https://jestjs.io/docs/getting-started

Learnt how to us jest for my CI tests here.

Task B: Improvements

Improvement 1

Frontend Input Validation:

Originally, the frontend of this accepted any text that has been entered into the input box and then sent it directly to the backend service, this meant that if someone was to input an empty input or multiple IP addresses that were not separated by commas, it was still sent to the backend without telling the user what went wrong.

```
if(!items.includes(",")){
    displayOutput("ERROR: IP addresses must be seprated with commas");
    return;
}
```

The first addition was this code, it is quite simple and just checks that it includes a comma between values and if not then it will output an ERROR with a short description telling the user what went wrong.

```
if(data.error) {
    displayOutput("and ERROR was returned byt he service: "+data.error);
}
```

My second change was adding this line to basically say that if the backend sent back a JSON that includes and error, the show it to the user instead of just treating it like a normal output.

This is important as previously the frontend would always assume that the backends response was valid, leaving the user with black outputs or confusing results that don't help them understand where they went wrong.

Overall, these changes help improve the useability of this application and helps avoid unnecessary requests to the backend services.

Additionally, this allows the frontend to be seen as more reliable as it catches the error before sending it to the backend and relying on it entirely to catch any and all errors sent by the user.

References:

https://developer.mozilla.org/enUS/docs/Web/JavaScript/Reference/Global_Objects/String/includes

Improvement 2

External configuration for service endpoints:

In the original implementation of the frontend, the frontend contained all the backend URLs hardcoded into the index.html file.

This means that any time a new service moved location or was deployed the source code for the front end would have to be edited and then redeployed each and every time which is not ideal for maintainability and would become a real issue if the front end became more complex or more services were added.

To hopefully fix this problem I stored my service URLS into a separate config named services.json.

```
<script type="text/javascript">
let serviceEndpoints = {};

fetch("services.json")
.then(response => response.json())
.then(data => {
    serviceEndpoints = data;
    console.log("The service configurations have been loaded", serviceEndpoints);
});
```

The front end now loads the file when the page starts and retrieves the correct endpoint for the functions based on the button the user selects.

This lets me change the endpoints without changing any of the core logic inside my frontend index.

```
// Java function with my Total Valid IPs
function getTotalValidIPs() {
    serviceCall(serviceEndpoints.totalValid, "items");
}

// Python function for my Classify IP version
function classifyIPs() {
    serviceCall(serviceEndpoints.classifyip, "ip");
}
```

Additionally this change makes it easier for future implementations to be made, such as adding multiple endpoints for the same service or allowing different version of the services.

References:

https://developer.mozilla.org/enUS/docs/Learn_web_development/Core/Scripting/JSON

Improvement 3

Show a loading or processing bar when functions are loading:

Currently in our frontend, when we press a button, nothing happens visually for around 2-3 seconds until the frontend returns the results.

This can be a real issue because the user might be impatient and assume after a second or so that the button is broken and might press it multiple times trying to fix it sending multiple requests.

```
function showingLoading(){
    document.getElementById('output-text').value ="processing request";

}
```

This current code is used to directly update the text area that displays results, this fixes the problem of users not knowing if they pressed the button correctly or not as it should now provide I simple indicator of “processing request” which tells the user that something is happening reducing the need for them to press the button multiple times and to encourage them to wait.

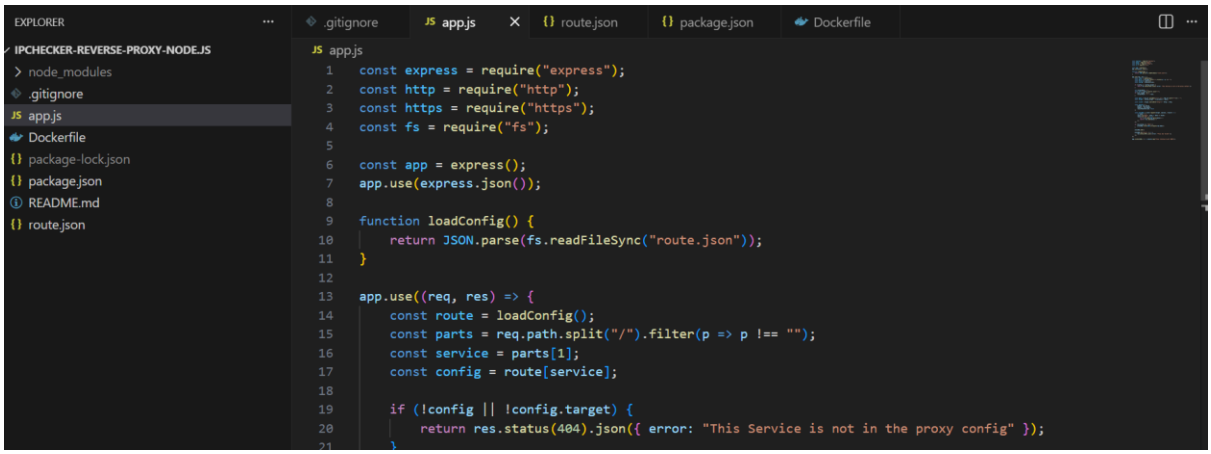
References:

https://developer.mozilla.org/en-US/docs/Web/API/Document/getElementById

Task C: Proxy

Implementation detailed(how you implemented, what it does, how does it work, etc):

I found express routing from to be quite useful for this so this is where I decided to make this service using Node.js as most of the examples were coded in this language, and I had previously used Node.js for coding my BadIp checker function in Task A, so I had a bit of experience with JavaScript files.



The image above shows my directory of files that I created as well as the start of my reverse proxy code.

As for how the Proxy works, the proxy forwards traffic to the containers stored on rancher.

The Proxy was configured to read service endpoint from a json file, I used a json file called route.json, which allowed backend target URLs to be changed , updated, or replaced without having to modify and of the source code.

```
"totalValid": {
  "target": "https://40407039.qpc.qubcloud.uk/totalvalid"
},
"classifyip": {
  "target": "https://40407039.qpc.qubcloud.uk/classifyip"
},
"countryInfo": {
  "target": "https://40407039.qpc.qubcloud.uk/countryinfo"
},
"badIp": {
  "target": "https://40407039.qpc.qubcloud.uk/badip"
},
"saverestoreSave": {
  "target": "https://40407039.qpc.qubcloud.uk/saverestore/save"
},
"saverestoreLoad": {
  "target": "https://40407039.qpc.qubcloud.uk/saverestore/load"
}
```

The Proxy would receive requests under /proxy/”Name of service” with name of service being what your service name was, and used URL parsing to match “Name of service” to a value corresponding to route.json.

I ensured that it supports both http and https as I previously had issues with google chrome where it was blocking certain services for not being secure then letting others through even though they weren’t secure.

A path normalisation change was added during development multiple times because the proxy would not work on certain functions for seemingly no reason.

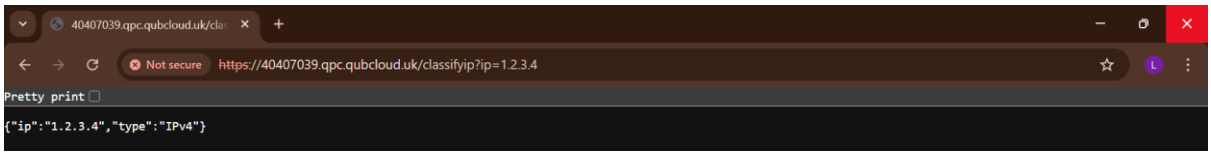
This lead to me figuring out that the functions it wasn't working on were all python services that utilised flask.

I then had to change the proxy so that it now only adds a slash when forwarding sub-paths, since flask seems to use whatever endpoint it feels like some days.

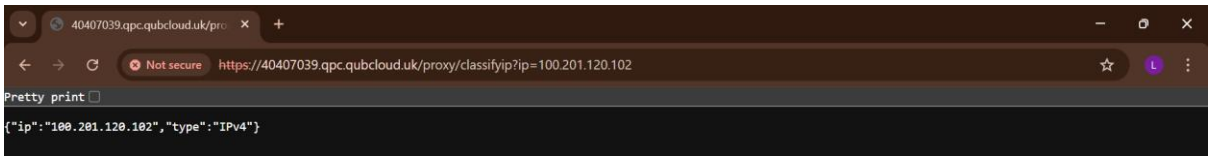
Testing Details(how you tested):

I tested it in a variety of ways:

Direct Backend calls, I directly entered the service URLs into my browser when I was getting proxy errors to ensure that it was the proxy that was the issue and that the original endpoints were not an issue.

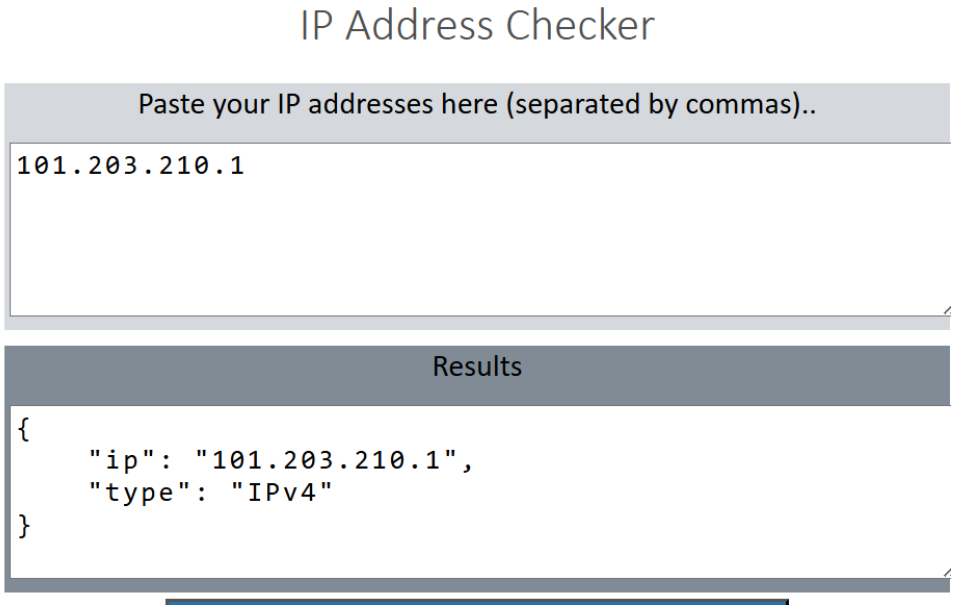


Once I confirmed the backend URLs still worked I moved onto testing the proxy calls for this, I ensured that I could call the services directly using /proxy just so I could make sure my reverse proxy code worked correctly.



As you can see this picture contains proxy in the URL confirming that my reverse proxy works in direct calls.

My next step was to test them with my frontend, this involved testing my 4 function buttons on the frontend as well as my save and load buttons.



Brief Review of success(did it work):

The reverse proxy successfully worked with all 4 of my function URLs and my frontend was able to access them all when loaded.

They all worked as expected and any errors or bugs were resolved in the end.

The design it proven to be scalable by allowing services to be integrated through entry to the route.json and the proxy fulfils its objectives.

Critical Thinking (300 words):

Overall microservice architecture helps provide a lot of flexibility, but can introduce issues, bugs and challenges when multiple entry points are utilised.

Without the use of some type of proxy, then each new function would have to be introduced with its own direct URL endpoint, leading to an increasingly more complicated and broken frontend with potentially duplicated logic throughout.

Proxy use can centralise routing and allow for the system to scale properly without overcomplicating the frontend.

The reverse proxy overall is not just routing, it safeguards your systems architecture and helps simplify development, reduces risk and helps boost overall system cohesion.

During development, I hit a pretty serious bug in my code where it would appear that the classification of my microservice has broken or wasn't working correctly as the browser kept outputting a syntax error for unexpected tokens, implying that it was expecting JSON but got HTML as a response.

I was convinced that this must be an error in my code and that I should rewrite the routing code or try implementing a bunch of fixes I saw online for similar problems.

After around 15 fixes and another 10 or so redeployments of my service I noticed something in common with the 3 buttons that were malfunctioning.

They all were coded in python and used flask and after some digging online I found that Flask can sometimes output endpoints that are different to my other languages I had used.

This lead to a much simpler fix than I had anticipated and overall demonstrated how some small issues in code can lead to very unpredictable errors and how I should have initially checked my failing functions to see what they had in common.

Anything else highlight(optional):

Ai such as Chat gpt were used in this project only for the purpose of clarifying error messages, helping to debug code and identifying issues in my code, it has not been used to directly generate or write the original code of my code in this assignment.

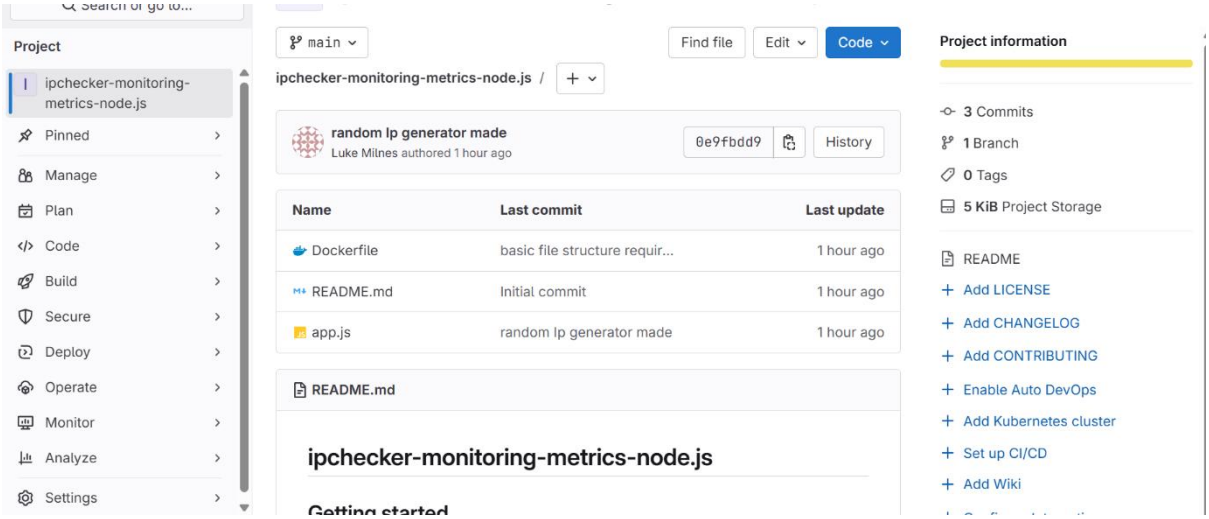
References:

https://expressjs.com/en/5x/api.html

I referred to this page quite a bit for learning how I could inspect URLs and forward them.

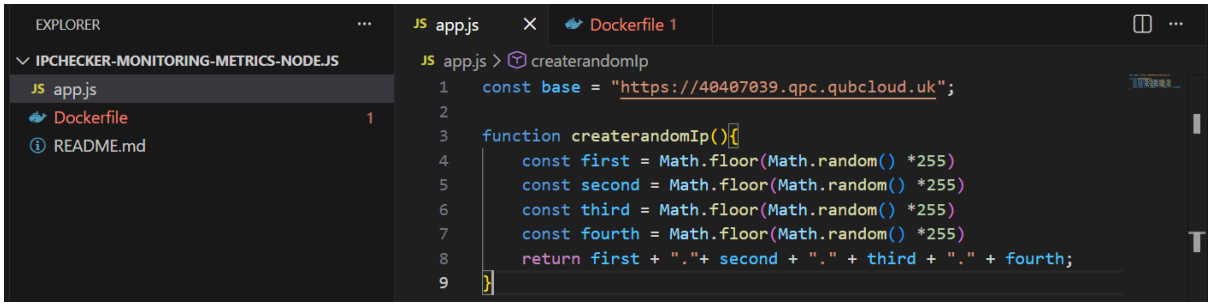
Task D: Monitoring

RepositoryURL: https://repository.hal.davecutting.uk/40407039/ipchecker-monitoring-metrics-node.js.git



Monitoring and metrics implemented and works:

I ran out of time to implement this section, this is all I have done



Critical thinking (300 words):

Anything to highlight(optional):

Ai such as Chat gpt were used in this project only for the purpose of clarifying error messages, helping to debug code and identifying issues in my code, it has not been used to directly generate or write the original code of my code in this assignment.

References:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/random

For making my random Ipv4 generator.

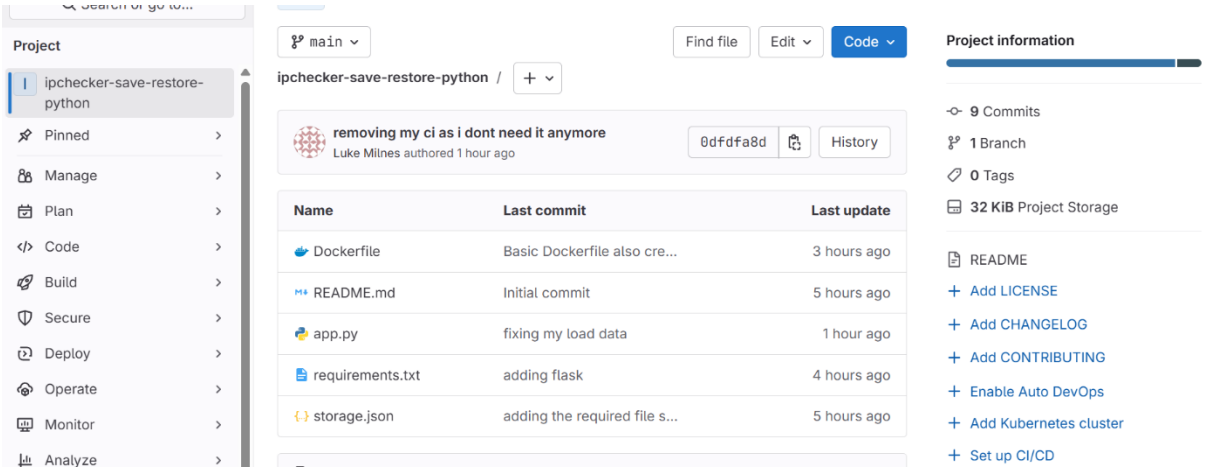
Task E: Stateful Saving

Overview:

I Have implemented stateful saving by creating a new repository which contains a dedicated microservice, responsible for securely storing then later retrieving IP address that the user has submitted.

This feature allows users to save a number of IP addresses during one session then later retrieve them by using a unique ID number.

Repository URL: https://repository.hal.davecutting.uk/40407039/ipchecker-save-restore-python.git



Live Service URLs:

https://40407039.qpc.qubcloud.uk/saverestore/save

https://40407039.qpc.qubcloud.uk/saverestore/load

Implementation:

The save and load service was developed using Python flask and was deployed separately as a containerised microservice.

It contained 2 main parts

POST save, this method excepts a JSON body container, adds the received test in a memory dictionary then returns a JSON response including a generated numeric ID

```
def saveData(data):
    with open(Storage_file, "w") as f:
        json.dump(data, f)

@app.route("/saverestore/save", methods=["POST"])
def save():
    inputData = request.get_json()
    text = inputData.get("text")

    if not text or text.strip() == "":
        return jsonify({"error":"no text sent"}), 400

    db = loadData()

    newId = len(db) + 1
    db[str(newId)] = text
    saveData(db)
```

And a Get load, that accepts a query parameter, looks up the saved IP list associated with the Id and returns the JSON container the original text if it is found or a relevant error if it is not found.

```
@app.route("/saverestore/load", methods=["GET"])
def load():
    id = request.args.get("id")
    db = loadData()

    if id not in db:
        return jsonify({"error": "id has not been found"}), 404

    return jsonify({"text": db[id]})
```

Frontend integration:

The existing frontend was extended to help communicate with the new microservice, this was done using JavaScript fetch calls.

I additionally went and added two new buttons, Save IP List and Load Saved Ips to frontend so the user would have easy access to these buttons.

```
<div>
    <button class="button-active" onclick="saveText();">Save IP List</button>
</div>
<div>
    <button class="button-active" onclick="loadText();">Load Saved IPs</button>
</div>
```

These buttons were placed so they would show up under the 4 main functions I previously added whilst still being above the clear button.

Additionally the service links used to call the backend have been placed inside the frontend for now, I was planning on having all of my ingress links being inside the

services.json improvement I made in Task B, but due to time constraints I just placed them in the script.

```
fetch("https://40407039.qpc.qubcloud.uk/saverestore/save", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ text: text })
```

```
fetch(`https://40407039.qpc.qubcloud.uk/saverestore/load?id=${id}`)
.then(r => r.json())
.then(data => {
    if (data.error) {
```

Docker:

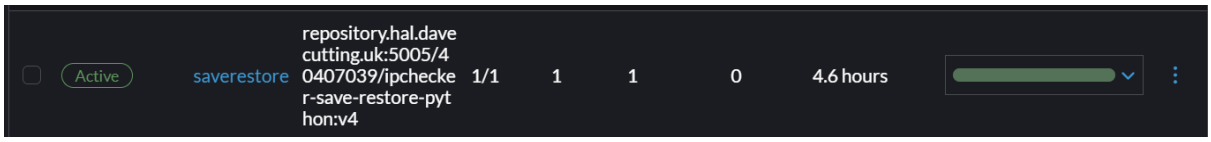
As with my previous Tasks functions, to deploy this python microservice in the Queens Private Cloud, a docker file would need to be creates to build a container image.

```
Dockerfile > ...
1   FROM python:3.12-slim
2
3   WORKDIR /app
4
5   COPY requirements.txt .
6   RUN pip install -r requirements.txt
7
8   COPY . .
9
10  EXPOSE 8080
11
12  CMD ["python", "app.py"]
```

This Dockerfile in particular was not hard to generate as it is very basic and quite similar to my function 2 Dockerfile from Task A.

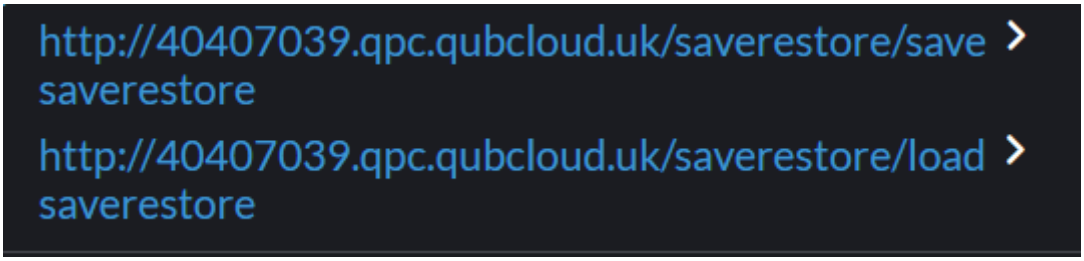
It copies over my requirements.txt, runs a pip install of my requirements file,which only contains flask and its version number.

Rancher:



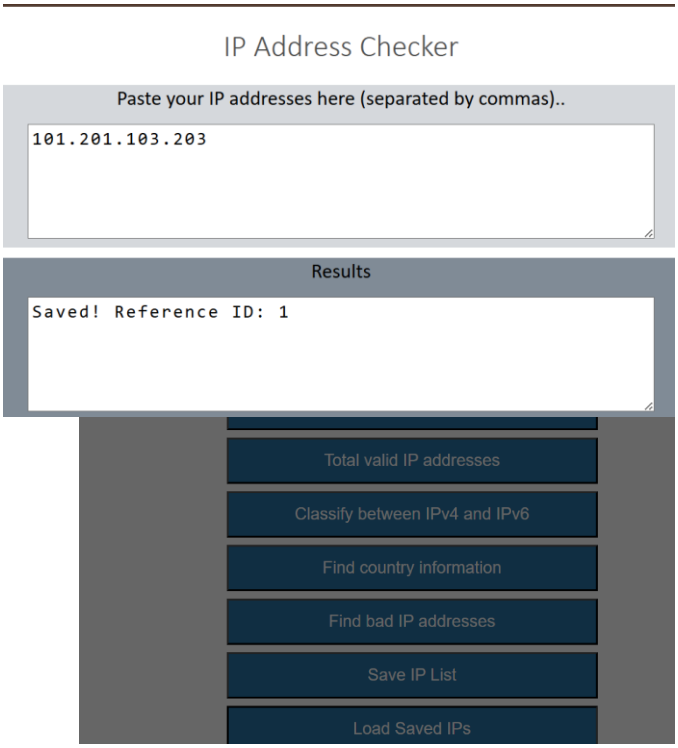
When it came to rancher, I had to upload and push my and containerise my docker image to my repository so that I could include this URL when I was creating my saverestor service.

Additionally I had to create two separate ingress links that connect to this service, save and load to allow those functions to call them in the frontend services.



Testing:

When it came to testing, I had the 2 core features I wanted to show off and get working, firstly was the save IP List button, this should let you input an IP address as shown on the right, click save, then Save the input and output a Saved! Reference ID: 1 Message to let the user know that it has been saved, so they don't spam click it and their ID that they should remember if they wish to get it back later.



After this has been done, if the user in the same session, wants to retrieve the IP addresses they previously stored then they would click on the Load Saved IP's button which will open a pop up menu from the service asking the user to input their ID.

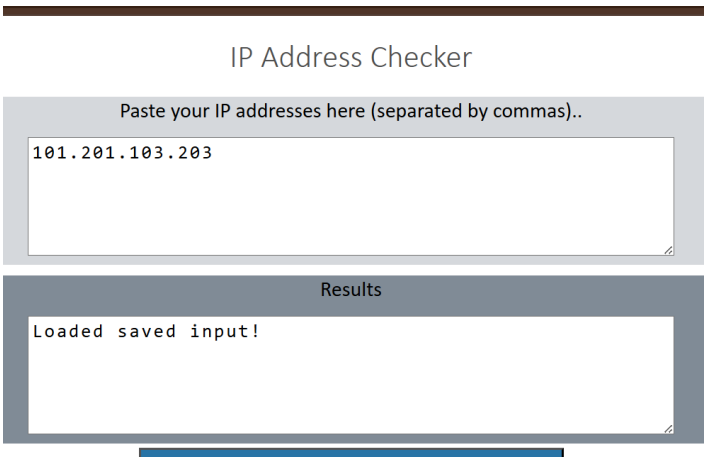
The user should remember their ID, in this example it is 1, and enter it in the given field then click ok.

This will then send the request to get that stored data and return it.

The third image to the right shows the output to the service, it will paste the IP previously saved in the Paste your IP addresses here section

And will let the user know that it has done so by outputting Loading saved input!.

Live functionality of this can be viewed during the video submission.



Anything to highlight:

Ai such as Chat gpt were used in this project only for the purpose of clarifying error messages, helping to debug code and identifying issues in my code , it has not been used to directly generate or write the original code in this assignment.

Any references are listed below:

https://flask.palletsprojects.com/en/stable/quickstart/#routing

https://flask.palletsprojects.com/en/stable/quickstart/#accessing-request-data

both used for routing my GET and POST methods

https://docs.python.org/3/library/json.html

How I read a wrote from a python library

https://docs.python.org/3/library/os.path.html

for open, read, writes and checking the file exists etc

Task F: Design

Include diagrams and text as appropriate demonstrating how your design would ensure high availability, fault tolerance, and resilience in the face of extreme disruptions – including hypothetical catastrophic events.

From what I understand of multicloud, is that multicloud is a cloud approach made up of more than 1 cloud service, from more than 1 cloud vendor-public or private.

Multiclouds have a variety of benefits, some of the main being Shadow IT, Flexibility, Proximity and Failover.

Companies normally choose to have multi cloud because they want to be able to get the best benefits from each provider.

Given the benefits of multicloud and the examples cloud providers given for this for this task, I would like to suggest using AWS, Azure and GCP for my multicloud IP checker application.

However, multicloud architecture does not also come without its negatives and challenges, especially when it comes to routing, data consistency and operation visibility.

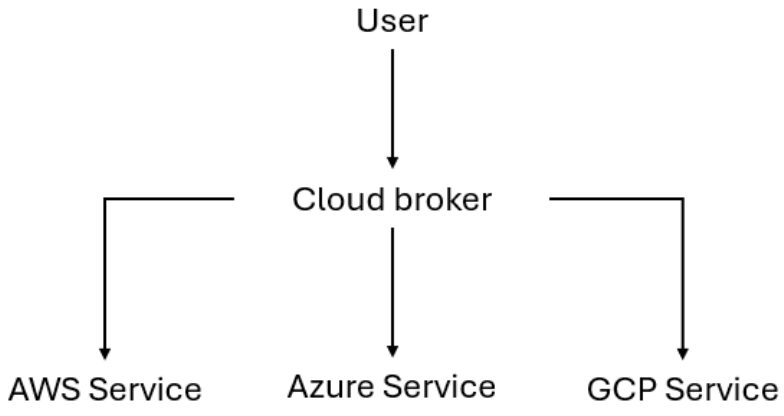
This is because from my knowledge each vendor will expose different load balancing technologies, managed services and routing which require centralised routing to coordinate traffic and requests to instances of cloud.

As mentioned previously I have decided to propose for my IP checker to run across AWS, Azure and GCP, supported by using a cloud broker.

The cloud broker should sit in front of all the cloud providers and holds the responsibility for ensuring traffic is directed to the correct cloud instance of my IP checker application overall unifying access and control.

The cloud broker would allow you to provision virtual machines on AWS, run data analytics on GCP and deploy containers on Azure all from one interface, it also helps enforce security policies and protect data across all cloud platforms

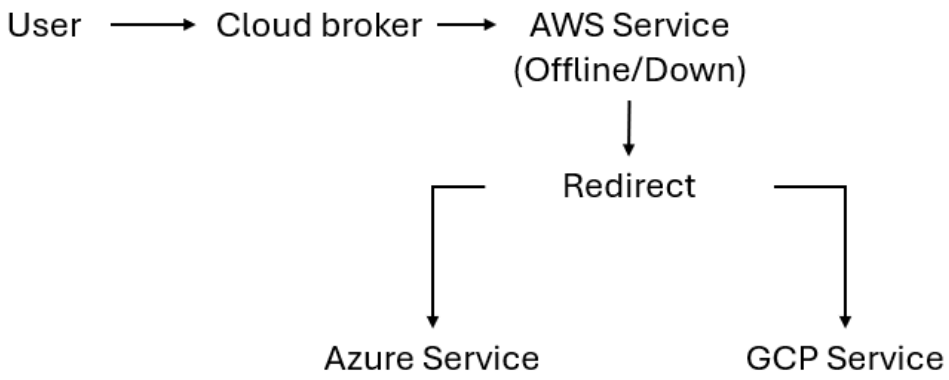
For my IP checker application I propose using this definition of a cloud broker. This broker would sit in front off all my cloud providers(AWS, Azure, GCP) and would be responsible for directing traffic to the correct IP checker service.



Example of high level multicloud architecture made in powerpoint.

The routing would monitor the general service health for each, the latency, load and would work around automatically rerouting traffic to another cloud if one of my main cloud becomes unreachable e.g. If My AWS and experiences a sudden outage, my

broker could reroute all the traffic to my Azure and GCP, without requiring any authorisation or changes.



Example of Failing cloud made in powerpoint.

In catastrophic scenarios such as earthquakes disrupting servers, or regional outages, this proposed architecture should ensure that the application should remain fully operational.

If AWS Services go down because out this outage, then the cloud broker would redirect the traffic and remove AWS from the routing zone, ensuring that all the traffic is redirect to the Azure and GCP Services as well, given they are available.

This would additionally be done without manual intervention.

References for Task F:

There was no use of AI tools such as chatgpt for this section, this work is my own expect for phrases and definitions I have taken from websites, but they should be referenced below:

https://www.redhat.com/en/topics/cloud-computing/what-is-multicloud?

“multicloud is a cloud approach made up of more than 1 cloud service, from more than 1 cloud vendor-public or private.”

https://youtu.be/twLPfr9j2ww?si=Kx2c7KyPtnjRcZfn

Used for helping me understand what a cloud broker does and how It works.

“ provision virtual machines on AWS, run data analytics on GCP and deploy containers on Azure all from one interface”

“helps enforce security policies and protect data across all cloud platforms”

(Time stamps for both quotes 1min 30 seconds)

All references used:

Task A

Function 1:

https://docs.oracle.com/javase/8/docs/jre/api/net/httpserver/spec/com/sun/net/httpserver/HttpServer.html

Server class I used

https://docs.oracle.com/javase/8/docs/jre/api/net/httpserver/spec/com/sun/net/httpserver/HttpExchange.html

used for my "exchange.getRequestMethod();" and my "exchange.getResponseBody();"

https://docs.oracle.com/javase/8/docs/api/java/lang/String.html#split-java.lang.String-

where I learned how to split ips

https://developer.mozilla.org/enUS/docs/Learn_web_development/Core/Scripting/JSON

helped with my manually built JSON

Function 2:

https://flask.palletsprojects.com/en/stable/quickstart/#routing

This website was used to help me understand how to properly route with flask.

https://flask.palletsprojects.com/en/stable/api/#module-flask.json

was used to help me understand how to use JSON with Flask

https://docs.python.org/3/library/stdtypes.html#str.split

where I got my idea for splitting strings in python.

https://flask.palletsprojects.com/en/stable/testing/

Used for when I was doing my CI tests

https://docs.python.org/3/reference/simple_stmts.html#the-assert-statement

Additionally referred to this in CI testing

Function 3:

https://learn.microsoft.com/en-us/aspnet/core/fundamentals/minimal-apis?view=aspnetcore-10.0

This is where I got some of my builder lines from for this function e.g. “var builder = WebApplication.CreateBuilder(args);” and ”var app = builder.Build();”

https://learn.microsoft.com/en-us/dotnet/api/system.string.split?view=net-10.0

where I got my string split logic from.

https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/async-scenarios

async for my CI testing

https://learn.microsoft.com/en-us/dotnet/api/system.net.http.httpclient?view=net-10.0

For making my http requests

Function 4:

https://expressjs.com/en/guide/routing.html

Used for express routing in my function.

https://expressjs.com/en/api.html#req.query

query parameters help.

https://developer.mozilla.org/enUS/docs/Learn_web_development/Core/Scripting/JSON

Outputting JSON.

https://jestjs.io/docs/getting-started

Learnt how to use jest for my CI tests here.

Task B

Improvement 1

https://developer.mozilla.org/enUS/docs/Web/JavaScript/Reference/Global_Objects/String/includes

Improvement 2

https://developer.mozilla.org/enUS/docs/Learn_web_development/Core/Scripting/JSON

Improvement 3

https://developer.mozilla.org/en-US/docs/Web/API/Document/getElementById

Task C

https://expressjs.com/en/5x/api.html

I referred to this page quite a bit for learning how I could inspect URLs and forward them.

Task D

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/random

For making my random Ipv4 generator.

Task E

https://flask.palletsprojects.com/en/stable/quickstart/#routing

https://flask.palletsprojects.com/en/stable/quickstart/#accessing-request-data

both used for routing my GET and POST methods

https://docs.python.org/3/library/json.html

How I read a wrote from a python library

https://docs.python.org/3/library/os.path.html

for open, read, writes and checking the file exists etc

Task F

https://www.redhat.com/en/topics/cloud-computing/what-is-multicloud?

“multicloud is a cloud approach made up of more than 1 cloud service, from more than 1 cloud vendor-public or private.”

https://youtu.be/twLPfr9j2ww?si=Kx2c7KyPtnjRcZfn

Used for helping me understand what a cloud broker does and how It works.

“ provision virtual machines on AWS, run data analytics on GCP and deploy containers on Azure all from one interface”

“helps enforce security policies and protect data across all cloud platforms”

(Time stamps for both quotes 1min 30 seconds)